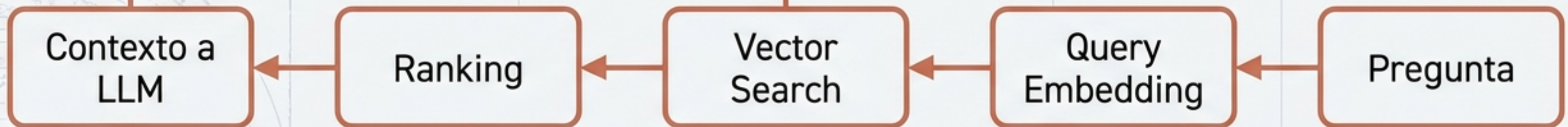
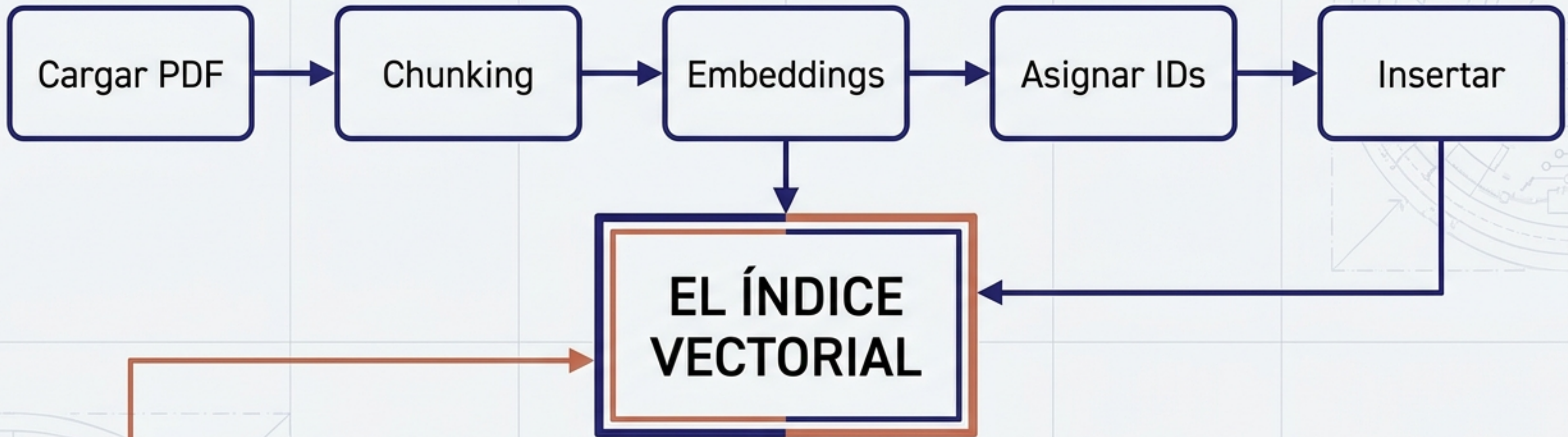


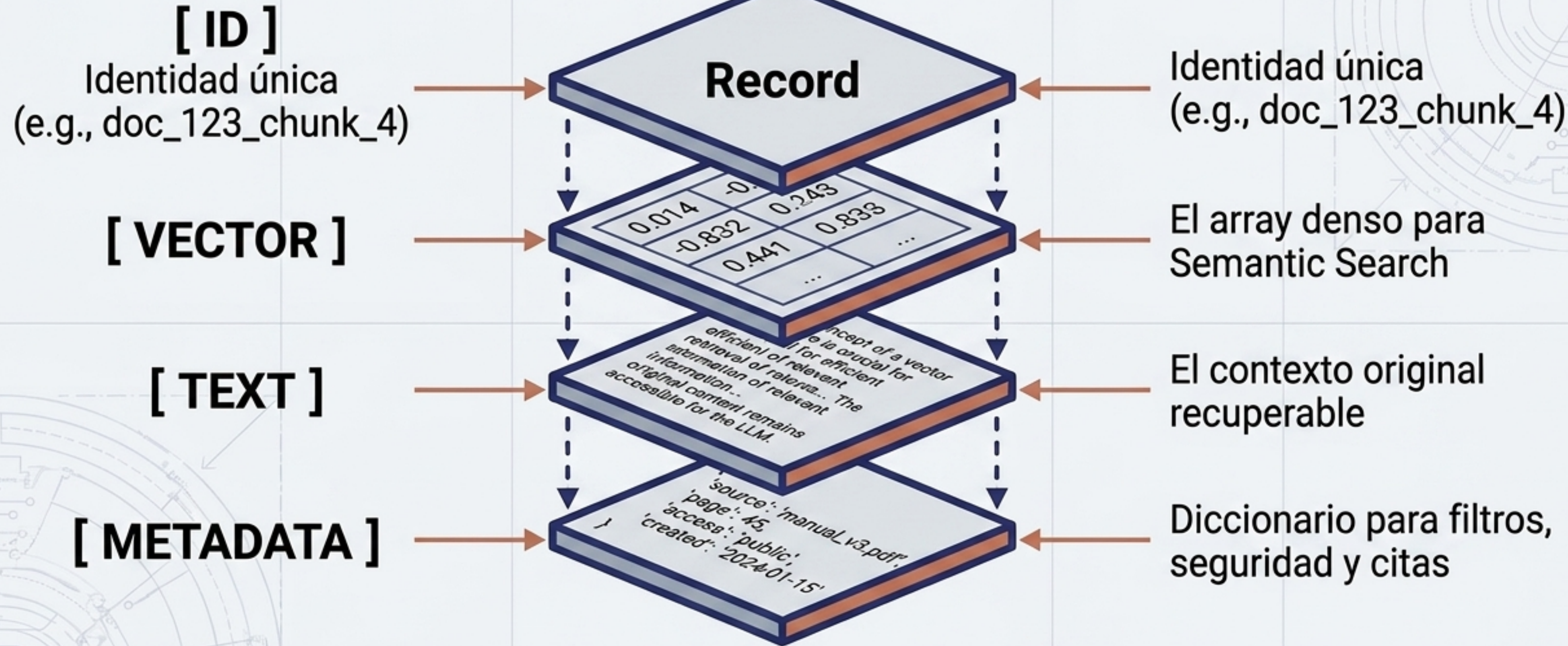
Pipeline de Indexación (Offline)



Pipeline de Consulta (Online)

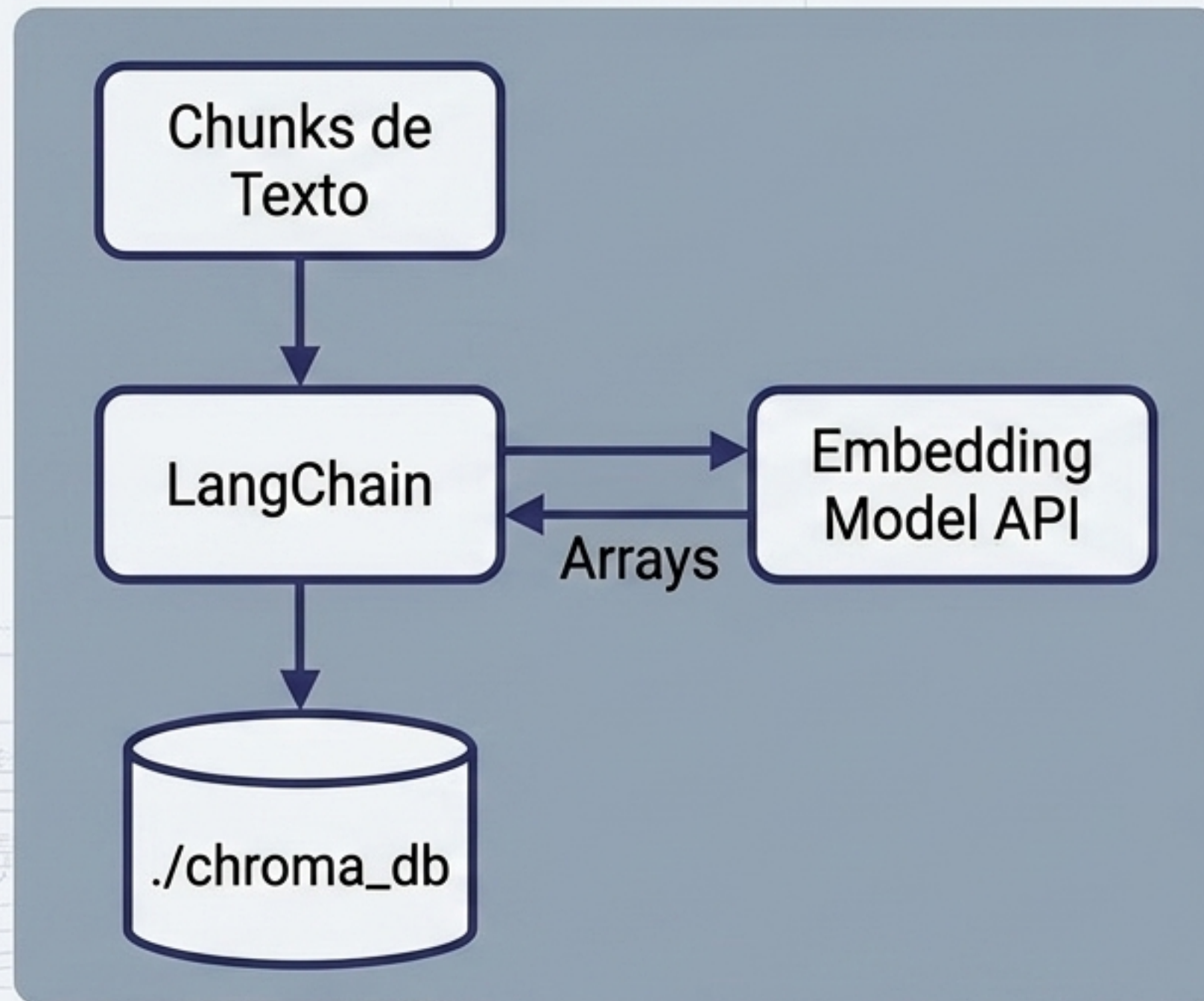
El LLM no consulta directamente la base vectorial. La aplicación hace retrieval primero y luego construye el contexto.

Ya tenemos arrays numéricos en memoria. ¿Dónde los guardamos?



El vector no reemplaza al documento. El embedding solo sirve para comparar; debemos conservar el texto original para enviarlo al LLM.

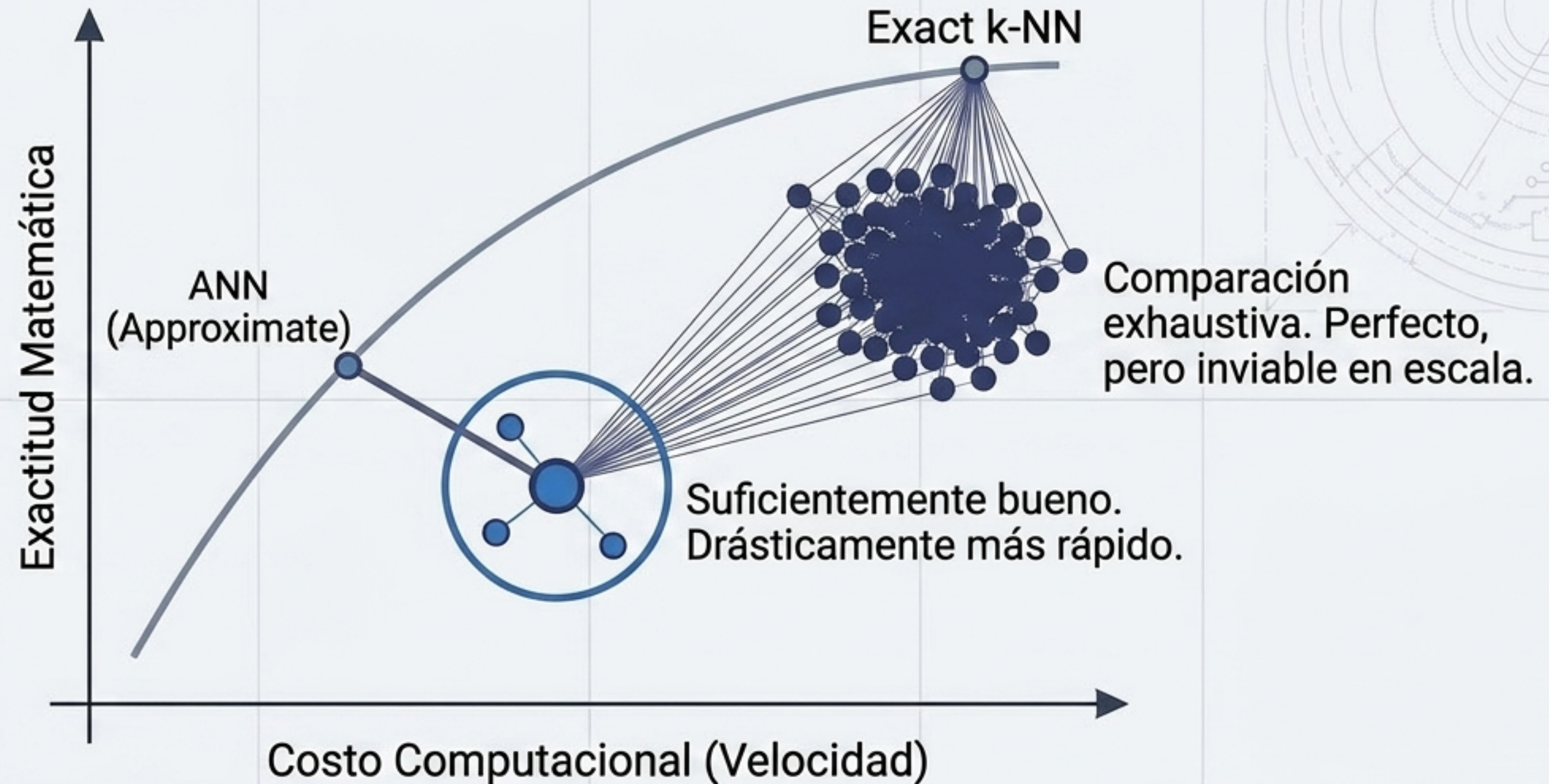
Introducción a Chroma con LangChain (Base orientada a aplicaciones)



```
1 from langchain_chroma import Chroma
2
3 vector_store = Chroma.from_documents(
4     documents=chunks,
5     embedding=embeddings,
6     persist_directory='./chroma_db'
7 )
```

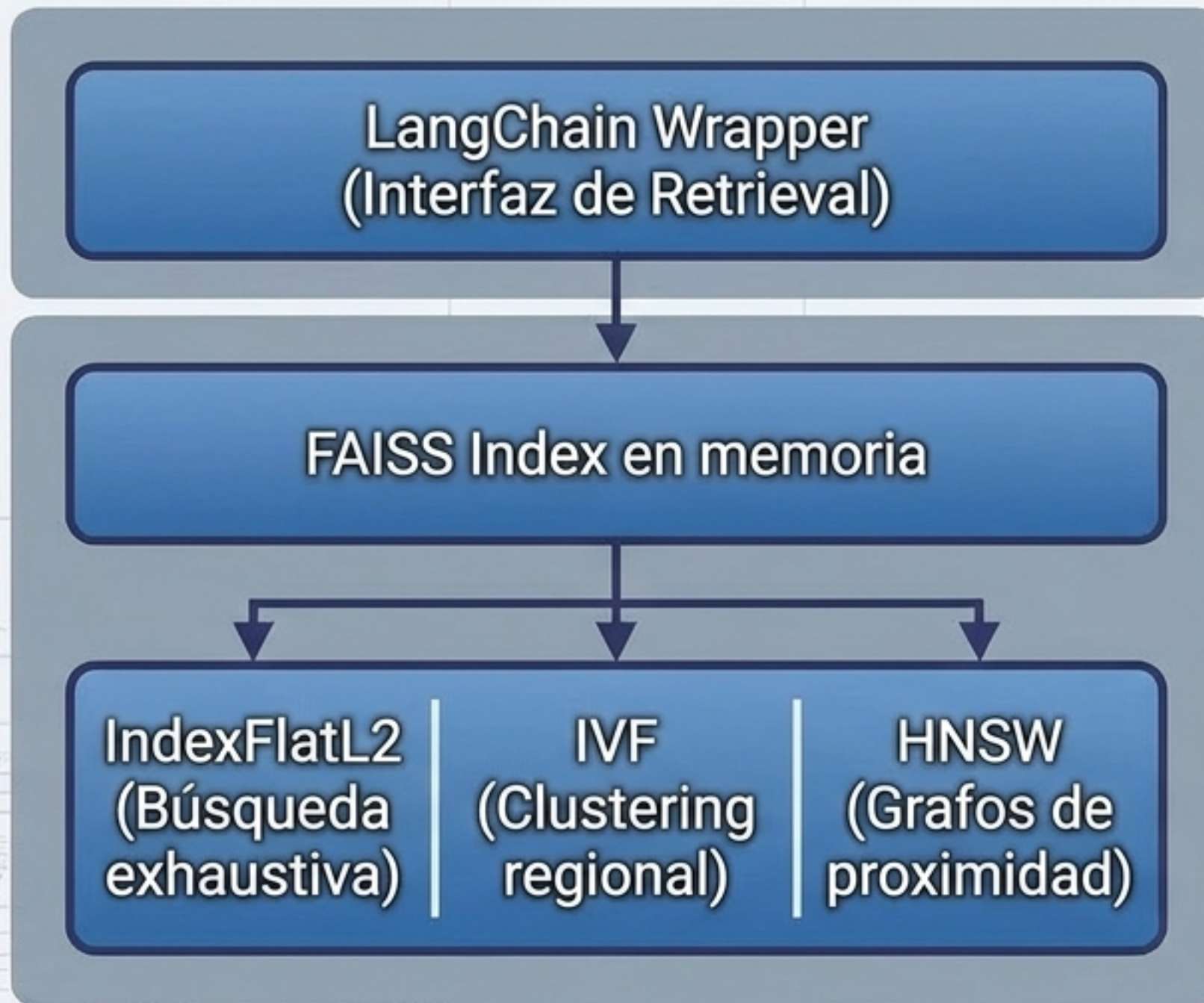
LangChain abstrae la complejidad: llama al modelo de embeddings y empaqueta el vector, el texto y la metadata.

¿Qué pasa con 10 millones de vectores? El Problema de Escala



El diseño del índice es crítico en producción. No buscamos el vecino perfecto, sino una aproximación eficiente.

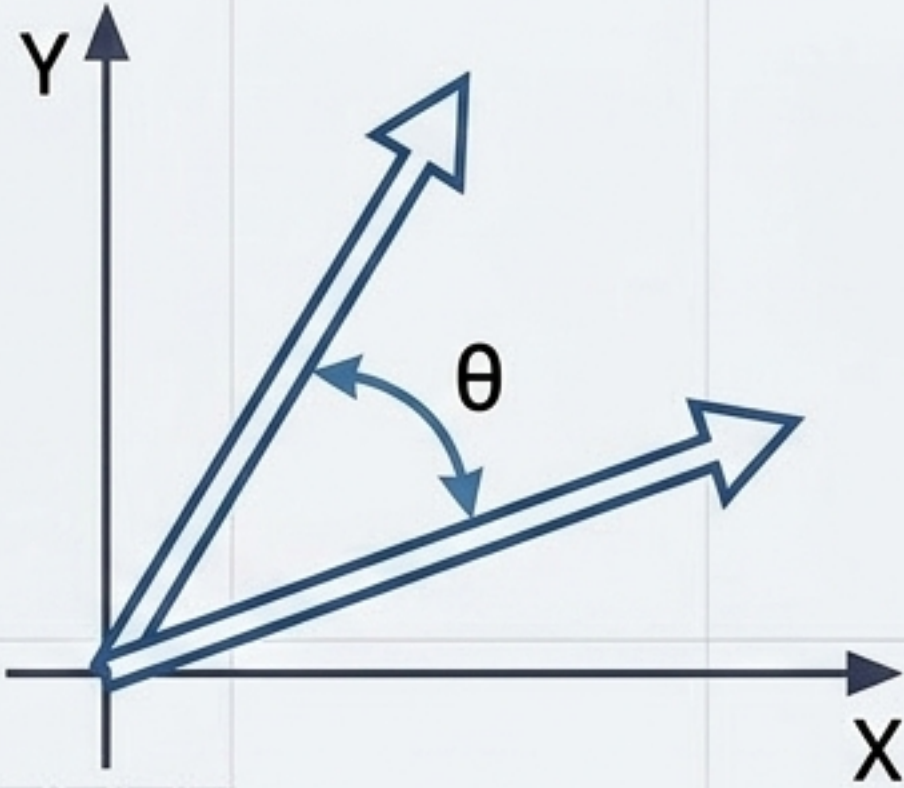
FAISS y el Control del Índice (Meta AI Research)



```
1 from langchain_community.vectorstores
2 import FAISS
3
4 # Creación con índice IndexFlatL2 oculto
  por defecto
5 vector_store = FAISS.from_documents(
6 chunks, embeddings)
7
8 resultados = vector_store.similarity_
  search('políticas de viaje')
```

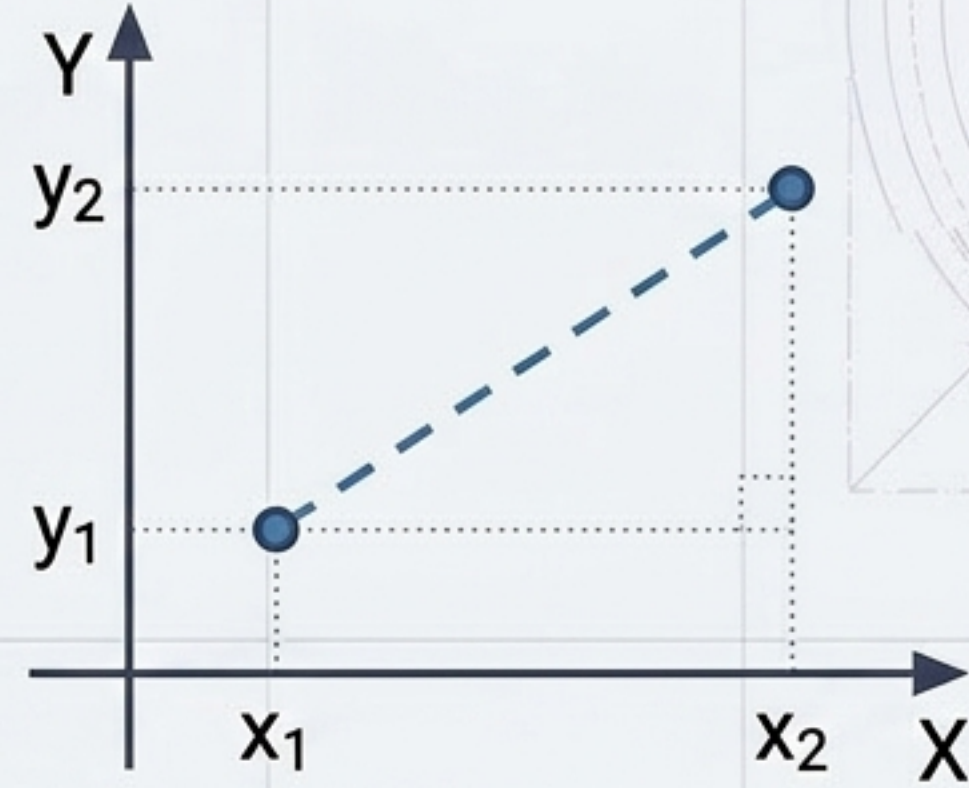
A diferencia de Chroma, FAISS no tiene persistencia automática.
Es puramente un motor matemático algorítmico.

Métricas: ¿Cómo sabe el índice qué vectores están cerca?



Cosine Similarity

- Mide la orientación y el ángulo entre vectores.
- Valores cercanos a 1 indican mayor similitud.



L2 / Euclidean Distance

- Mide la separación espacial absoluta entre dos puntos.
- Valores cercanos a 0 indican menor distancia (mejor).

Distance (separación espacial) no es lo mismo que Similarity (cercanía de orientación).

Diagnosticando la Trampa del Score

FAISS ×

```
Backend: FAISS (L2 Distance)  
Query: 'políticas de viaje'  
similarity_search_with_score()  
-> Score: 0.15
```



(Excelente resultado)

Chroma ×

```
Backend: Chroma (Cosine Similarity)  
Query: 'políticas de viaje'  
similarity_search_with_score()  
-> Score: 0.15
```



(Terrible resultado)

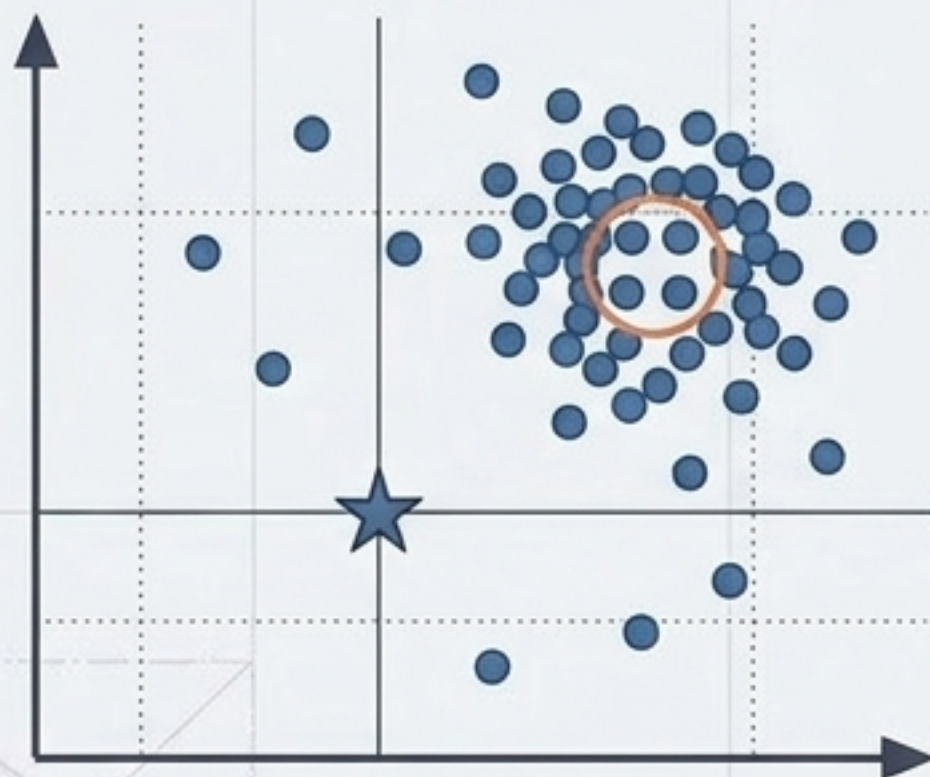
¡Peligro! Nunca copies un threshold (e.g., score > 0.75) ciegamente entre proyectos.

Un backend puede devolver **distancia** (menor es mejor), mientras otro devuelve **similitud** (mayor es mejor).

Antes de interpretar un score, pregunta: ¿Es **distancia** o **similitud**?

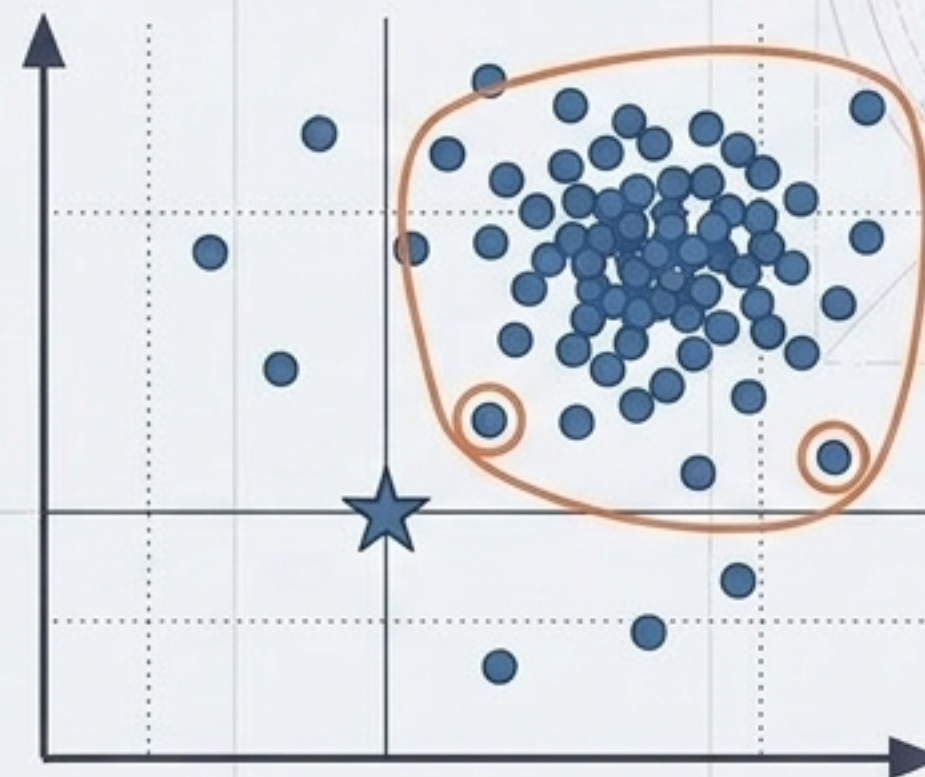
Diversidad en la Recuperación: El Problema de la Redundancia (MMR)

Búsqueda Top-K



Selecciona los 4 vecinos más cercanos.
Resultado: Alta relevancia, pero a menudo respuestas idénticas y redundantes.

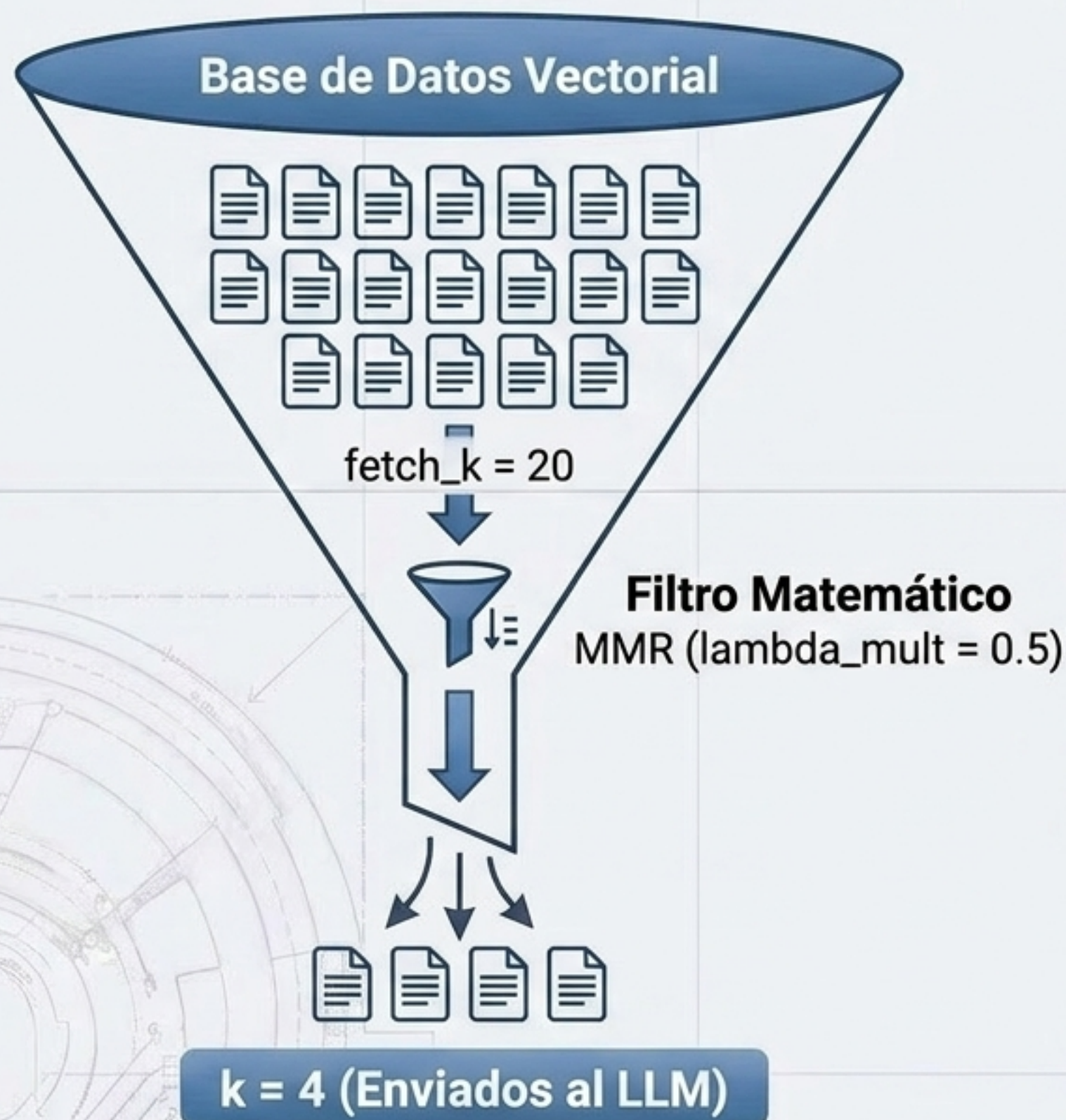
MMR (Diversidad)



Selecciona resultados esparcidos por la zona de relevancia.
Resultado: Respuestas variadas que cubren más contexto.

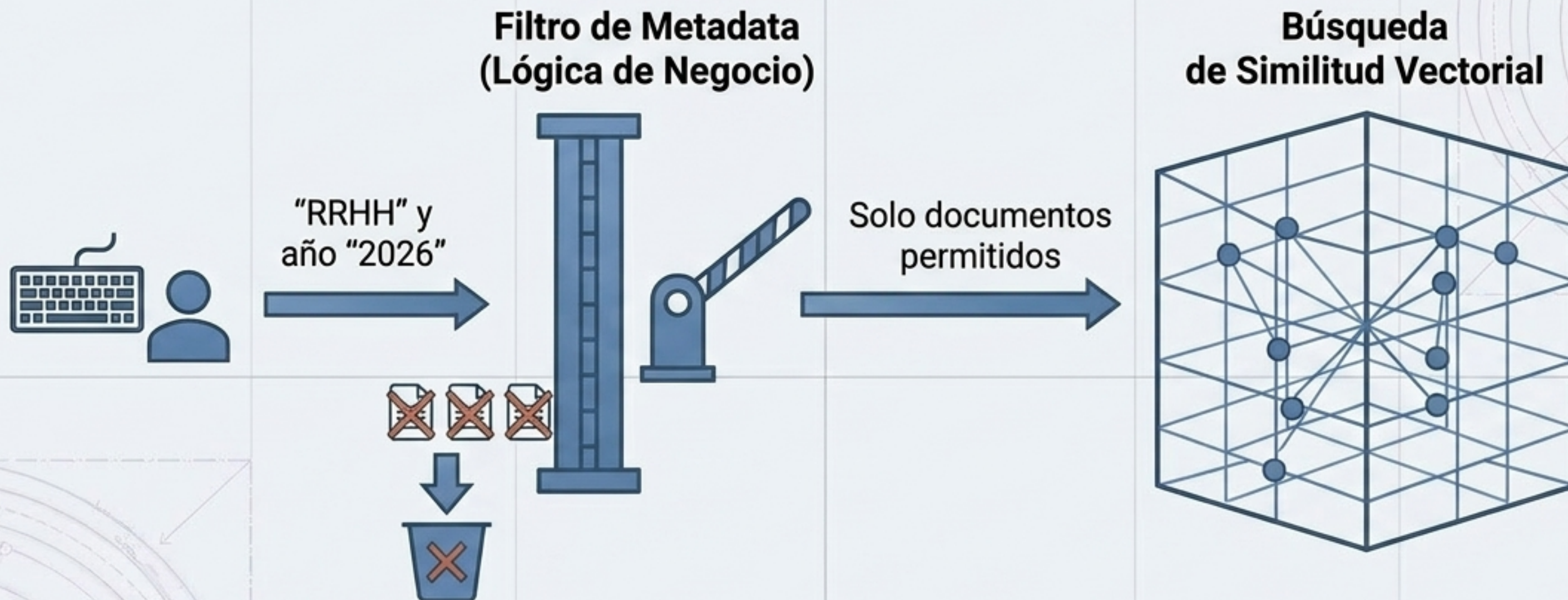
**Lógica MMR = Relevancia con la Query
MENOS Redundancia con lo ya seleccionado**

El Retriever de LangChain (Abstracción de Política)



```
Python Code ×  
  
# vector_store es el backend.  
Retriever es la política.  
retriever = vector_store.as_retriever(  
    search_type='mmr',  
    search_kwargs={  
        'k': 4,  
        'fetch_k': 20,  
        'lambda_mult': 0.5  
    }  
)  
  
documentos = retriever.invoke('¿Cómo  
pedir vacaciones?')
```


Restricciones Estructuradas (Metadata Filtering)

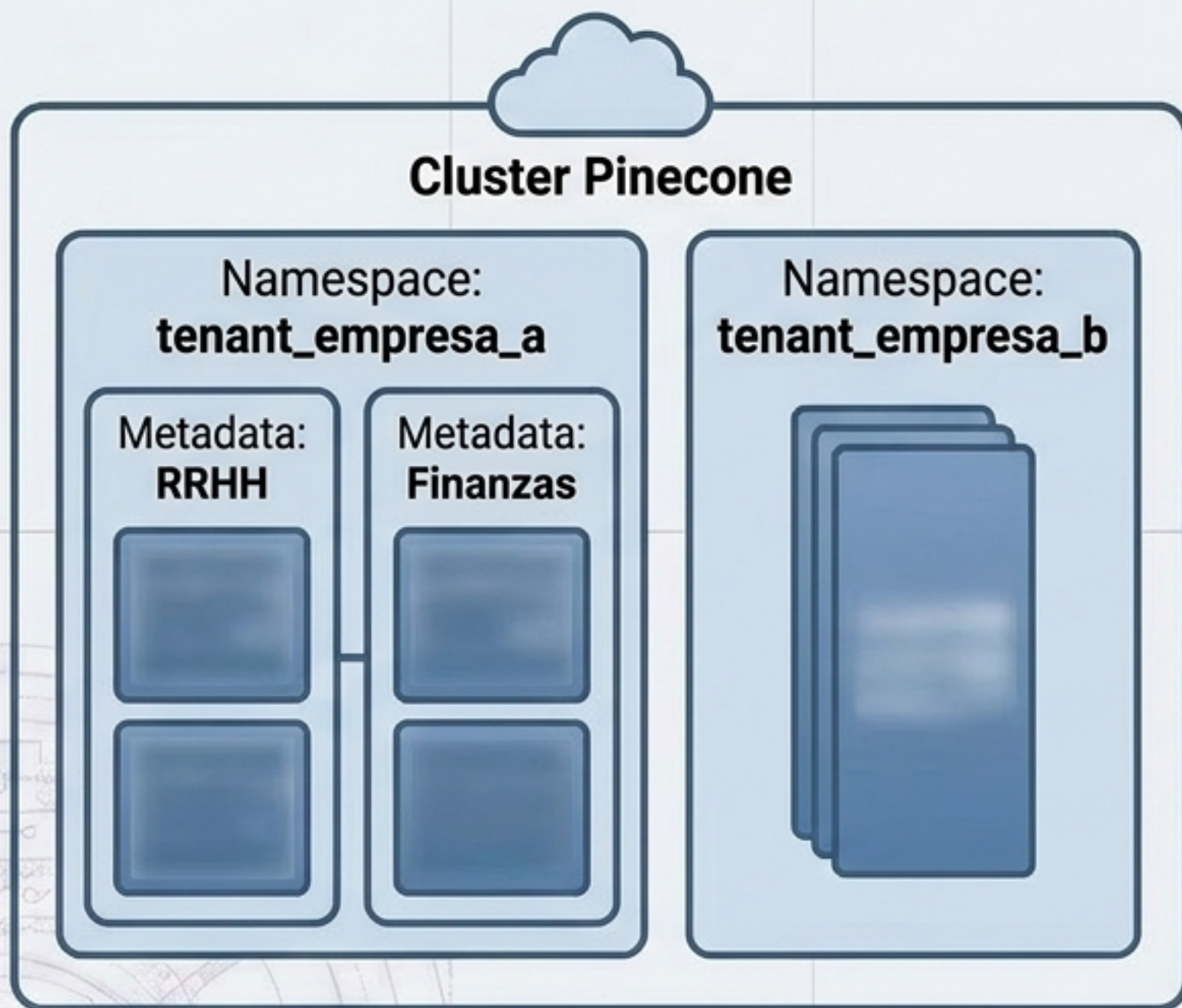


Vector similarity responde: ¿Qué contenido se parece semánticamente?
Metadata filtering responde: ¿Qué conjunto de documentos está permitido que participe en la búsqueda?



Los permisos deben aplicarse **ANTES** o **DURANTE** el retrieval. Nunca confíes en que el LLM censurará información confidencial que ya fue recuperada.

Pinecone y la Nube (Infraestructura SaaS Administrada)



Namespaces separan físicamente los datos (Multitenancy).
Metadata filtra dentro de un mismo pool de datos.

```
Python Code ×  
  
from langchain_pinecone import  
PineconeVectorStore  
  
retriever = vector_store.as_retriever(  
    search_kwargs={  
        'filter': {'departamento':  
            'RRHH'},  
        'namespace': 'tenant_empresa_a'  
    }  
)
```


El Ciclo de Vida: IDs, Idempotencia y Reindexación

Actualizar
Documentos



Se requieren **IDs determinísticos** (ej. hash de la URL + índice de chunk) para permitir upserts y evitar duplicar información al actualizar.

Idempotencia



Ejecutar el pipeline de indexación dos veces no debe duplicar el tamaño de la base de datos. Los IDs idénticos sobrescriben el contenido.

Cambio de
Modelo



Pasar de 'text-embedding-ada-002' a 'text-embedding-3-small' vuelve incompatibles los espacios vectoriales.



La Regla de Oro de la Indexación:

Si cambias el modelo de embeddings, debes recrear el índice desde cero.

Matriz de Decisión Arquitectónica

	Chroma	FAISS	Pinecone
Principal Enfoque	Aplicaciones IA locales y prototipos rápidos.	Similarity local, experimentación y pipelines offline.	Producción Cloud SaaS y escalabilidad masiva.
Ventaja Clave	Facilidad de uso, colecciones integradas, persistencia local.	Control total algorítmico sobre ANN (IndexFlat, IVF, HNSW).	Multitenancy nativo (Namespaces), infraestructura administrada.
La Frase Docente	Nos enseña el concepto de base vectorial.	Nos enseña el problema matemático de búsqueda vectorial.	Nos enseña cómo consumir vector search como infraestructura.

Evaluación de Aislamiento

Módulo Retriever

- Hit Rate
- MRR
- nDCG

Módulo LLM

- Fluency
- Hallucination

“La generación no puede corregir información que nunca fue recuperada. La calidad del Retriever establece el techo de calidad del sistema RAG.”

Checklist de Producción

- ✓ Evaluar el retrieval **independientemente** del LLM.
- ✓ Calibrar los **thresholds** usando métricas **reales** (no copiar el 0.75).
- ✓ Separar tenants usando **namespaces rígidos** o **filtros estrictos**.